SHETH L.U.J & SIR MV COLLEGE

Aim:Digital Signaature

implement digital signature algorithms such as RSA-based signatures, and verify the integrity and authenticity of digitally signed messages.

Code:CLI

```
import hashlib
import secrets
import math

def is_prime(n, k=10):
    if n < 2:
        return False
    small_primes = [2, 3, 5, 7, 11, 13, 17, 19, 23, 29]
    for p in small_primes:
        if n == p:
            return True
        if n % p == 0:
            return False

    d = n - 1
    r = 0
    while d % 2 == 0:
        r += 1
        d //= 2

    for _ in range(k):
        a = secrets.randbelow(n - 3) + 2
        x = pow(a, d, n)

        if x == 1 or x == n - 1:
            continue

        for _ in range(r - 1):
            x = pow(x, 2, n)
            if x == n - 1:
                break
        else:
```

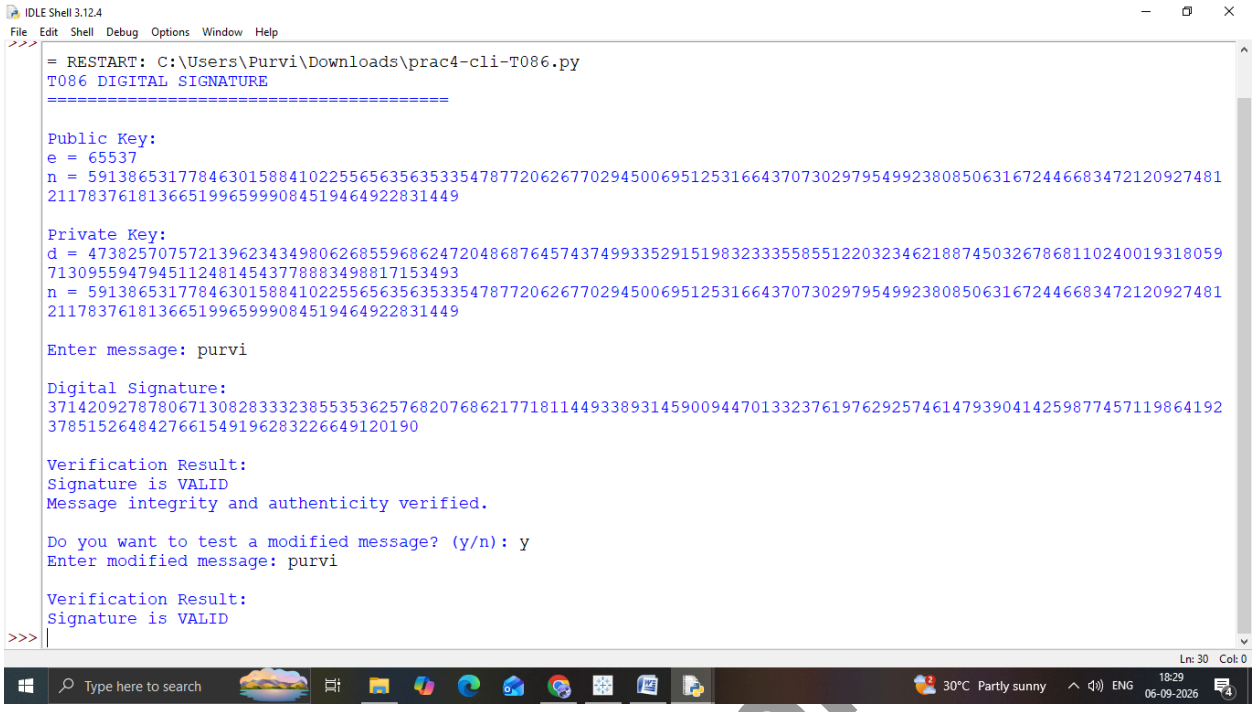
```
            return False

    return True

def generate_prime(bits):
    while True:
        n = secrets.randbits(bits)
        n |= (1 << bits - 1) | 1
        if is_prime(n):
            return n

def generate_keys():
    p = generate_prime(256)
    q = generate_prime(256)

    while p == q:
        q = generate_prime(256)

    n = p * q
    phi = (p - 1) * (q - 1)

    e = 65537

    while math.gcd(e, phi) != 1:
        e += 2

    d = pow(e, -1, phi)

    public_key = (e, n)
    private_key = (d, n)

    return public_key, private_key

def hash_message(message):
    return int.from_bytes(
        hashlib.sha256(message.encode()).digest(),
        byteorder="big"
    )

def sign_message(message, private_key):
    d, n = private_key
    h = hash_message(message)
    signature = pow(h, d, n)
    return signature

Purvi.R.Karkera
T086
```

```
def verify_signature(message, signature, public_key):
    e, n = public_key
    h = hash_message(message)
    recovered_hash = pow(signature, e, n)
    return h == recovered_hash

def main():
    print("T086 DIGITAL SIGNATURE")
    print("=" * 40)

    public_key, private_key = generate_keys()

    print("\nPublic Key:")
    print("e =", public_key[0])
    print("n =", public_key[1])

    print("\nPrivate Key:")
    print("d =", private_key[0])
    print("n =", private_key[1])

    message = input("\nEnter message: ")

    signature = sign_message(message, private_key)

    print("\nDigital Signature:")
    print(signature)

    result = verify_signature(message, signature, public_key)

    print("\nVerification Result:")
    if result:
        print("Signature is VALID")
        print("Message integrity and authenticity verified.")
    else:
        print("Signature is INVALID")

    choice = input("\nDo you want to test a modified message? (y/n): ")

    if choice.lower() == "y":
        modified_message = input("Enter modified message: ")

        result = verify_signature(
            modified_message,
```

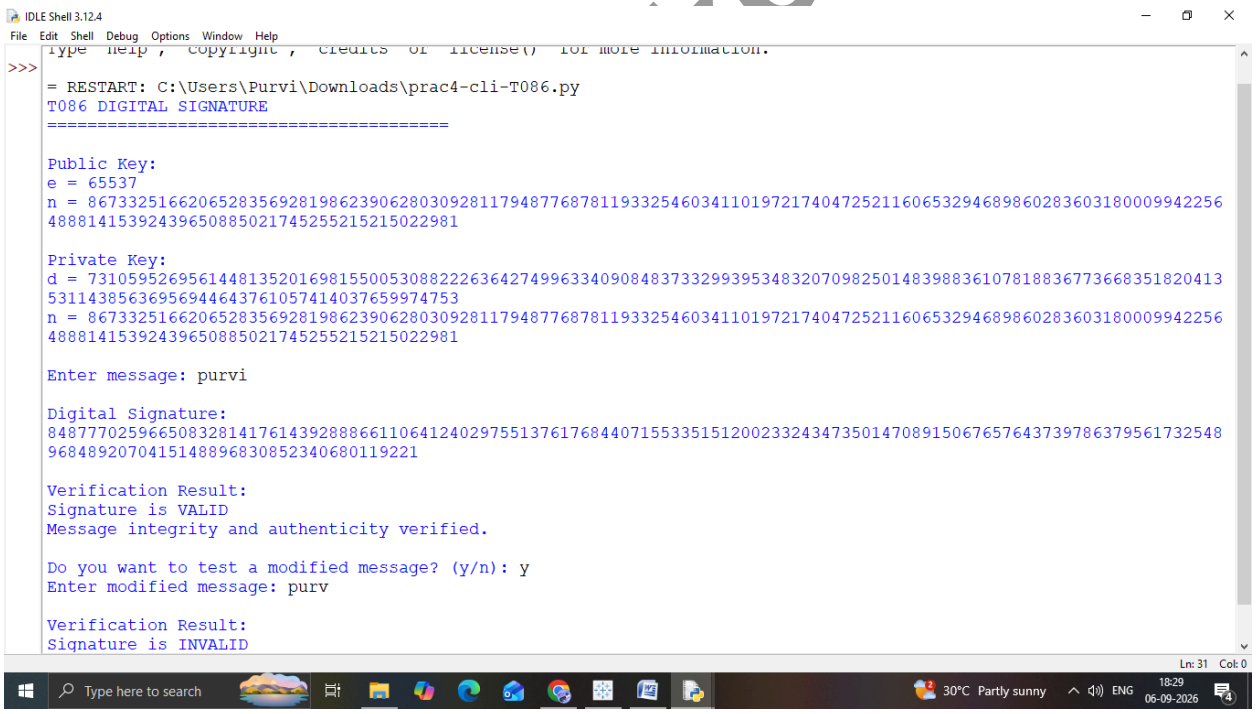
```
        signature,
        public_key
    )

    print("\nVerification Result:")
    if result:
        print("Signature is VALID")
    else:
        print("Signature is INVALID")
        print("Message has been modified.")

if __name__ == "__main__":
    main()
```

Output:valid

SHETH L.U.J & SIR MV COLLEGE



invalid



Purvi.R.Karkera
T086

SHETH L.U.J & SIR MV COLLEGE

Code:GUI

```
import tkinter as tk
from tkinter import messagebox
import hashlib
import secrets
import math

def is_prime(n, k=10):
    if n < 2:
        return False

    small_primes = [2, 3, 5, 7, 11, 13, 17, 19, 23, 29]

    for p in small_primes:
        if n == p:
            return True
        if n % p == 0:
            return False

    d = n - 1
    r = 0

    while d % 2 == 0:
        r += 1
        d //= 2

    for _ in range(k):
        a = secrets.randbelow(n - 3) + 2
        x = pow(a, d, n)

        if x == 1 or x == n - 1:
            continue

        for _ in range(r - 1):
            x = pow(x, 2, n)

            if x == n - 1:
                break
        else:
            return False

    return True
```

Purvi.R.Karkera
T086

```
def generate_prime(bits):
    while True:
        n = secrets.randbits(bits)
        n |= (1 << bits - 1) | 1

        if is_prime(n):
            return n

def generate_keys():
    p = generate_prime(256)
    q = generate_prime(256)

    while p == q:
        q = generate_prime(256)

    n = p * q
    phi = (p - 1) * (q - 1)

    e = 65537

    while math.gcd(e, phi) != 1:
        e += 2

    d = pow(e, -1, phi)

    return (e, n), (d, n)

def hash_message(message):
    return int.from_bytes(
        hashlib.sha256(message.encode()).digest(),
        "big"
    )

def sign_message(message, private_key):
    d, n = private_key
    h = hash_message(message)
    return pow(h, d, n)

def verify_signature(message, signature, public_key):
    e, n = public_key
    h = hash_message(message)
    recovered_hash = pow(signature, e, n)
    return h == recovered_hash
```

Purvi.R.Karkera
T086

```
public_key = None
private_key = None
signature = None

def generate():
    global public_key, private_key, signature

    public_key, private_key = generate_keys()
    signature = None

    public_key_box.delete("1.0", tk.END)
    private_key_box.delete("1.0", tk.END)
    signature_box.delete("1.0", tk.END)

    public_key_box.insert(
        tk.END,
        f"e = {public_key[0]}\nn = {public_key[1]}"
    )

    private_key_box.insert(
        tk.END,
        f"d = {private_key[0]}\nn = {private_key[1]}"
    )

    result_label.config(
        text="Keys generated successfully",
        fg="green"
    )

def sign():
    global signature

    if private_key is None:
        messagebox.showerror(
            "Error",
            "Generate keys first."
        )
        return

    message = message_box.get("1.0", tk.END).rstrip()

    if not message:
        messagebox.showerror(
```

```
            "Error",
            "Enter a message."
        )
        return

    signature = sign_message(message, private_key)

    signature_box.delete("1.0", tk.END)
    signature_box.insert(tk.END, str(signature))

    result_label.config(
        text="Message signed successfully",
        fg="green"
    )

def verify():
    if public_key is None:
        messagebox.showerror(
            "Error",
            "Generate keys first."
        )
        return

    message = message_box.get("1.0", tk.END).rstrip()
    signature_text = signature_box.get("1.0", tk.END).strip()

    if not message:
        messagebox.showerror(
            "Error",
            "Enter a message."
        )
        return

    if not signature_text:
        messagebox.showerror(
            "Error",
            "Generate a signature first."
        )
        return

    try:
        signature_value = int(signature_text)
    except ValueError:
        messagebox.showerror(
```

```
            "Error",
            "Invalid signature."
        )
        return

    if verify_signature(
        message,
        signature_value,
        public_key
    ):
        result_label.config(
            text="VALID SIGNATURE\nMessage integrity and authenticity verified",
            fg="green"
        )
    else:
        result_label.config(
            text="INVALID SIGNATURE\nMessage has been modified or signature is invalid",
            fg="red"
        )

root = tk.Tk()
root.title("T086-Digital Signature")
root.geometry("800x700")
root.configure(bg="#f2f2f2")

title = tk.Label(
    root,
    text="T086-DIGITAL SIGNATURE",
    font=("Arial", 22, "bold"),
    bg="#f2f2f2",
    fg="#222222"
)
title.pack(pady=15)

message_label = tk.Label(
    root,
    text="Message",
    font=("Arial", 13, "bold"),
    bg="#f2f2f2"
)
message_label.pack()

message_box = tk.Text(
    root,
```

```
    height=5,
    width=85,
    font=("Arial", 11)
)
message_box.pack(pady=5)

button_frame = tk.Frame(
    root,
    bg="#f2f2f2"
)
button_frame.pack(pady=10)

generate_button = tk.Button(
    button_frame,
    text="Generate Keys",
    command=generate,
    width=18,
    bg="#3498db",
    fg="white",
    font=("Arial", 11, "bold")
)
generate_button.grid(row=0, column=0, padx=5)

sign_button = tk.Button(
    button_frame,
    text="Sign Message",
    command=sign,
    width=18,
    bg="#27ae60",
    fg="white",
    font=("Arial", 11, "bold")
)
sign_button.grid(row=0, column=1, padx=5)

verify_button = tk.Button(
    button_frame,
    text="Verify Signature",
    command=verify,
    width=18,
    bg="#8e44ad",
    fg="white",
    font=("Arial", 11, "bold")
)
verify_button.grid(row=0, column=2, padx=5)

Purvi.R.Karkera
T086
```

```
public_label = tk.Label(
    root,
    text="Public Key",
    font=("Arial", 12, "bold"),
    bg="#f2f2f2"
)
public_label.pack()

public_key_box = tk.Text(
    root,
    height=4,
    width=85,
    font=("Arial", 9)
)
public_key_box.pack(pady=5)

private_label = tk.Label(
    root,
    text="Private Key",
    font=("Arial", 12, "bold"),
    bg="#f2f2f2"
)
private_label.pack()

private_key_box = tk.Text(
    root,
    height=4,
    width=85,
    font=("Arial", 9)
)
private_key_box.pack(pady=5)

signature_label = tk.Label(
    root,
    text="Digital Signature",
    font=("Arial", 12, "bold"),
    bg="#f2f2f2"
)
signature_label.pack()

signature_box = tk.Text(
    root,
    height=5,
```

```
    width=85,
    font=("Arial", 9)
)
signature_box.pack(pady=5)

result_label = tk.Label(
    root,
    text="Generate keys to begin",
    font=("Arial", 12, "bold"),
    bg="#f2f2f2",
    fg="blue"
)
result_label.pack(pady=15)

root.mainloop()
```


Output: